

PROJECT REPORT ON

Task Synchronization and CPU Utilization

Analysis for Sensor Readings

A dissertation submitted in partial fulfillment of the requirements for the
award of the degree of

MASTER OF TECHNOLOGY

In

EMBEDDED SYSTEMS

By

KULSUM SHIREEN

Roll Number: 22011D5520

Under the Esteemed Guidance of

Dr. M. ASHA RANI

Professor, Department of ECE

UNIVERSITY COLLEGE OF ENGINEERING, SCIENCE & TECHNOLOGY,

HYDERABAD



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING, SCIENCE AND TECHNOLOGY,
HYDERABAD.

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD-500085

2024

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING, SCIENCE & TECHNOLOGY HYDERABAD
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD-500085



CERTIFICATE BY THE SUPERVISOR

This is to certify that the project entitled “**Task Synchronization and CPU Utilization Analysis for Sensor Readings**” submitted by **KULSUM SHIREEN** bearing Roll No. **22011D5520** in partial fulfillment for the award of the degree of **MASTER OF TECHNOLOGY** in **EMBEDDED SYSTEMS** in the Department of Electronics and Communication Engineering, University College of Engineering, Science & Technology Hyderabad, Jawaharlal Nehru Technological University, during the academic year 2022-2024 is a record of bonafide work carried out by her under my guidance and supervision.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Project Supervisor

Dr. M. ASHA RANI

Professor in ECE

Department of Electronics and Communication Engineering
University College of Engineering, Science & Technology Hyderabad

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING, SCIENCE & TECHNOLOGY HYDERABAD
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD-500085



CERTIFICATE BY THE HEAD OF THE DEPARTMENT

This is to certify that the project entitled “**Task Synchronization and CPU Utilization Analysis for Sensor Readings**” is a bonafide work done by **KULSUM SHIREEN** bearing the Roll No. **22011D5520** in partial fulfillment for the award of the degree of **MASTER OF TECHNOLOGY** in **EMBEDDED SYSTEMS** in the Department of Electronics and Communication Engineering, University College of Engineering, Science & Technology Hyderabad, Jawaharlal Nehru Technological University, during the academic year 2023-2024.

Dr. A. RAJANI

Professor & Head of ECE Department

Department of Electronics and Communication Engineering

University College of Engineering, Science & Technology Hyderabad

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING, SCIENCE & TECHNOLOGY HYDERABAD
(A)

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD-500085



DECLARATION

I hereby declare that the project work entitled “**Task Synchronization and CPU Utilization Analysis for Sensor Readings**” which is being submitted to the Jawaharlal Nehru Technological University Hyderabad, in partial fulfillment of the requirements for the award of the Degree of **MASTER OF TECHNOLOGY** in **EMBEDDED SYSTEMS**, Department of Electronics and Communication Engineering is a bonafide report of the work carried out by me. The material contained in this Report has not been submitted to any University or Institution for the award of any degree or diploma.

KULSUM SHIREEN

Roll Number: 22011D5520

Department of Electronics and Communication Engineering
University College of Engineering, Science & Technology Hyderabad

Place: Hyderabad

Date:

ACKNOWLEDGEMENT

This is an acknowledgment of the intensive drive and technical competence of many individuals who have contributed to the success of my project.

With great pleasure I would like to express my deep sense of gratitude and sincere thanks to my project guide **Dr. M. ASHA RANI**, Professor, Department of ECE, JNTUHCEH (A), Her timely advice, meticulous scrutiny, scholarly advice, and scientific approach have been indispensable to bring about the successful completion of my project and would also like to thank **Mrs. R. Dhruva**, Research Scholar who has provided constant support and helped us rectify our mistakes.

It is my privilege to thank the Principal **Dr. K VIJAYA KUMAR REDDY**, for providing us with an opportunity to do this project.

I would like to express my sincere thanks to **Dr. A. RAJANI**, Professor, Head of the Department, Department of ECE, JNTUHCEH (A) for her valuable suggestions and support.

I would like to thank my Project Review Committee **Dr. L. PRATAP REDDY and Dr. M. ASHA RANI** for their insightful comments and constant encouragement.

Finally, I would also like to thank the Almighty, my family, and my friends for their constant encouragement and support in carrying out my project work.

KULSUM SHIREEN
22011D55020

Table of Content

Abstract.....	6
1. Introduction.....	7
2. Literature Review.....	14
3. Implementation and Evaluation of Task Synchronization and CPU Utilization.....	15
4. Proposed Methodology for Task Synchronization.....	19
5. Sensor Reading With Application Of Semaphore And Observing CPU Utilization Time.....	23
6. RESULTS.....	29
7. Conclusion and FutureScope.....	31
Appendix.....	32

ABSTRACT

Sensor networks, especially those using devices like the ESP32-S3, are essential for collecting and monitoring data. However, one of the main challenges is ensuring that tasks within these networks are synchronized accurately. Our research focuses on this issue by examining how using semaphores—tools that help manage task coordination—affects synchronization accuracy, CPU usage, and the system's ability to grow as needed.

We propose a system that uses semaphores to manage task coordination. These semaphores ensure that only one task can access shared resources at a time, preventing conflicts and errors when multiple tasks try to operate simultaneously. By using semaphores, we can ensure that tasks run in an organized manner, leading to reliable data collection.

To test our ideas, we conducted experiments comparing scenarios with and without semaphores, focusing on both single-core and dual-core setups. We measured CPU usage in situations where synchronization tools were used and where they were removed. This study provides practical recommendations for effectively coordinating tasks in ESP32-S3-based sensor networks.

CHAPTER-1

Introduction

An embedded system is a specialized computing architecture designed to perform dedicated functions within a larger mechanical or electrical system. Unlike general-purpose computing devices, embedded systems are optimized for specific tasks, offering efficiency in performance, power consumption, and cost. These systems are ubiquitous in modern technology, found in applications ranging from consumer electronics like smart appliances and mobile devices to critical industrial machinery and automotive control systems. Typically comprising a combination of hardware and firmware, embedded systems leverage microcontrollers or microprocessors to execute predetermined operations while operating in real-time constraints. Their design emphasizes reliability, responsiveness, and resource optimization, making them integral to advancements in automation, IoT (Internet of Things), and smart technologies. As the demand for intelligent and interconnected devices continues to grow, the study and development of embedded systems are becoming increasingly vital in engineering and technology fields.

1.1 Real Time Embedded System

A real-time embedded system is a specialized computing system designed to perform specific tasks within strict timing constraints. These systems are essential in applications where timely processing and response are critical, such as in automotive control systems, industrial automation, medical devices, and telecommunications. Real-time embedded systems can be classified into hard real-time systems, which must meet stringent deadlines to avoid catastrophic failures, and soft real-time systems, where occasional deadline misses are tolerable but can affect performance. These systems utilize dedicated hardware and software tailored to ensure reliable and deterministic behavior, making them integral to various industries that rely on precise timing and control.

1.2 Synchronization

In real-time embedded systems, synchronization is vital for ensuring that tasks operate smoothly and reliably within their strict timing constraints. As systems often involve multiple concurrent tasks that may share resources, proper synchronization helps to prevent conflicts that could lead to data corruption, race conditions, or deadlocks. These issues can severely disrupt the system's operation, potentially causing missed deadlines or even catastrophic failures in

critical applications such as automotive safety systems or medical devices.

Synchronization mechanisms, such as mutexes, semaphores, and condition variables, play a crucial role in coordinating task execution. For instance, mutexes ensure that only one task can access a shared resource at a time, while semaphores can signal when a resource is available or when a task can proceed. This prevents tasks from interfering with each other and maintains data integrity.

Moreover, effective synchronization allows for predictable behavior in real-time systems, which is essential for meeting timing requirements. By controlling the order in which tasks are executed, developers can ensure that high-priority tasks are completed on time, reducing the risk of deadline misses. In summary, synchronization is a key component in the design of real-time embedded systems, enabling them to function correctly and reliably in dynamic environments where timing and resource management are critical.

1.3 Types of Tasks in Real-Time Embedded Systems

Real-time embedded systems manage a variety of tasks, each with distinct characteristics and requirements. Understanding these types is crucial for effective task synchronization and CPU utilization analysis. Here are five key types of tasks:

1.3.1. Periodic Tasks

Periodic tasks are characterized by their execution at regular, fixed intervals. These tasks are essential for applications requiring consistent and predictable data updates or control actions. The timing of periodic tasks is often governed by their periodicity, defined by a cycle time, which dictates how frequently the task must run. Their deterministic behavior allows for effective scheduling strategies, making them fundamental to maintaining system reliability.

1.3.2. Aperiodic Tasks

Aperiodic tasks do not follow a fixed schedule and can be triggered by unpredictable events or conditions. Their irregular nature means that they may need to be executed as soon as possible when the event occurs. Aperiodic tasks can introduce challenges in scheduling, as their execution can interfere with the timing of periodic tasks. Effective handling of aperiodic tasks is essential for maintaining overall system performance and responsiveness.

1.3.3. Sporadic Tasks

Sporadic tasks are similar to aperiodic tasks but include a minimum inter-arrival time constraint, ensuring that they do not occur too frequently. This constraint allows the system to manage sporadic tasks more effectively, as they can be scheduled within certain bounds. The need for timely execution of sporadic tasks often arises from critical event responses, making it important to balance their execution with other tasks to avoid system overload.

1.3.4. Soft Real-Time Tasks

Soft real-time tasks are those where timing constraints are important but not absolute. While these tasks strive to meet their deadlines, occasional misses do not lead to system failure. They are commonly found in applications like multimedia processing, where timely data delivery enhances performance but is not critical. Managing soft real-time tasks requires careful scheduling to optimize resource utilization while ensuring acceptable performance levels.

1.3.5. Hard Real-Time Tasks

Hard real-time tasks have stringent timing requirements, where missing a deadline can result in severe consequences, including system failure or safety hazards. These tasks are critical in applications such as automotive control systems and medical devices, where precise timing is essential for safety and functionality. Effective scheduling and synchronization of hard real-time tasks are paramount to ensuring their timely execution, often requiring the use of priority-based scheduling algorithms.

Each type of task in real-time embedded systems presents unique challenges and considerations for synchronization and CPU utilization. By understanding these task types, we can develop more effective strategies for managing resource allocation and ensuring system reliability, particularly in applications involving sensor readings and data collection.

1.4 Microcontroller: ESP32-S3

The **ESP32-S3** microcontroller was selected for this project because of its advanced features, making it highly suitable for **task synchronization** and **CPU utilization analysis** in real-time embedded systems. The following attributes demonstrate why the ESP32-S3 is the ideal choice:

1.4.1. Dual-Core Architecture: The ESP32-S3 contains two **Xtensa LX7 cores**, allowing it to handle **parallel task execution** efficiently. In this project, sensor readings from devices like the **MAX30100** (for pulse oximetry) and **DHT11** (for temperature and humidity) are processed simultaneously, reducing the risk of task overlaps or conflicts. The ability to allocate tasks to separate cores improves overall system responsiveness and ensures that real-time data collection tasks are completed without delays or interference.

1.4.2. FreeRTOS Integration for Task Management: The ESP32-S3 is built with **FreeRTOS** support, a real-time operating system that facilitates **multi-tasking** and **task synchronization**. FreeRTOS provides crucial tools like **semaphores**, **task priorities**, and **mutexes** that help manage concurrent tasks, preventing issues like **data corruption** or **race conditions** when multiple tasks access shared resources. In this project, semaphores are used to ensure orderly execution of tasks, ensuring the sensors function smoothly without conflicting with each other or causing delays in data processing.

1.4.3. Optimized CPU Utilization: Efficient use of the ESP32-S3's dual-core processor, along with the task synchronization mechanisms provided by FreeRTOS, helps to **reduce CPU utilization**. By distributing tasks across both cores and ensuring that resources are accessed in a coordinated manner, the system minimizes unnecessary CPU usage, thus optimizing overall performance. This feature is especially important in real-time systems, where the timely processing of sensor data is crucial.

1.4.4. Low Power Consumption: The ESP32-S3 is designed for **low-power IoT** applications, making it an excellent choice for systems that need to operate on minimal power. By employing task synchronization techniques that reduce CPU load and avoid task contention, the microcontroller conserves energy. This is vital for battery-powered

devices, ensuring that the system remains operational for longer periods without frequent recharging.

1.4.5. Scalability and Integrated Communication: The ESP32-S3 also comes equipped with **Wi-Fi** and **Bluetooth** connectivity, which makes it suitable for a wide range of **IoT** and **sensor network** applications. These communication features allow the system to easily scale, supporting the addition of new sensors or devices while maintaining reliable performance. The integrated communication ensures that sensor data can be transmitted wirelessly to remote systems for further processing, without burdening the CPU.

The **ESP32-S3** is an ideal microcontroller for this project due to its **dual-core processing**, **real-time task management capabilities** through FreeRTOS, **efficient CPU utilization**, **low power operation**, and **scalable communication options**. These attributes ensure that the system can handle real-time sensor data collection effectively while maintaining reliability, making it suitable for **task synchronization** and **CPU utilization analysis** in embedded sensor networks.



Figure 1.1: ESP32S3 Microcontroller

1.5 ESP-IDF: Development Framework for ESP32-S3

The **ESP-IDF (Espressif IoT Development Framework)** is the official development platform for the **ESP32-S3** microcontroller, providing essential software tools, libraries, and APIs for application development. Key features that contribute to task synchronization and CPU utilization analysis include:

1.5.1. Comprehensive Toolchain: ESP-IDF offers a complete toolchain, including compilers, debuggers, and flashing tools, simplifying the development process for real-time applications. The framework supports **xTaskCreate** for task creation, enabling developers to assign task priorities and manage stack sizes efficiently.

1.5.2. Real-Time Operating System Support: Integrated with **FreeRTOS**, ESP-IDF facilitates efficient task management through synchronization primitives like **semaphores** and **mutexes**. This ensures coordinated access to shared resources, preventing data corruption and allowing reliable sensor data acquisition.

1.5.3. Multi-Core Support: The framework leverages the ESP32-S3's dual-core architecture, allowing tasks to be executed in parallel. This capability optimizes CPU utilization, enhancing overall system performance by distributing workloads effectively.

1.5.4. Power Management Features: ESP-IDF includes advanced power management functionalities that help reduce energy consumption. By utilizing low-power states, the framework ensures efficient operation, which is crucial for battery-powered IoT devices.

1.5.5. Network Connectivity: With built-in support for **Wi-Fi** and **Bluetooth**, ESP-IDF enables seamless integration of the ESP32-S3 into IoT networks. This allows for reliable data transmission from sensors to remote systems, ensuring real-time accessibility.

ESP-IDF is a critical framework that supports efficient development and deployment of real-time applications on the **ESP32-S3**. Its comprehensive toolchain, real-time operating system support, multi-core capabilities, power management features, and integrated communication options collectively enhance task synchronization and CPU utilization in embedded sensor networks.



Figure 1.2:Espressif Framework

CHAPTER-2

Literature Review

Tinotenda Zwavashe et al. (2014) implemented a task synchronization technique using RTOS on a PIC microcontroller, focusing on single-core systems. Their work highlighted the challenges of handling multiple tasks in a single-core environment, particularly in terms of CPU utilization and task conflicts. While they achieved functional synchronization, the lack of dual-core processing limited their ability to optimize CPU time and efficiently manage task execution. In contrast, our project builds on these findings by leveraging the dual-core ESP32-S3 microcontroller, which allows for more efficient task management, particularly with the use of semaphores to prevent task conflicts and reduce CPU time in a multi-core setup.

Sanjeev Khushu and Johnathan Simmons (2001) provided a theoretical explanation of the importance of task scheduling and task synchronization in real-time embedded systems. Their research underscored the critical role these techniques play in ensuring system reliability and performance. While their work focused on the theoretical aspects, our project puts these principles into practice by implementing task synchronization using semaphores in a real-time system for sensor data acquisition. By applying their insights, we were able to design a system that optimizes task execution, minimizes CPU utilization, and ensures reliable sensor readings in a dual-core, real-time environment.

CHAPTER-3

Implementation and Evaluation of Task Synchronization and CPU Utilization

This chapter discusses the creation of tasks for data acquisition from the MAX30100 pulse oximeter and the DHT11 temperature and humidity sensor. The ESP32-S3 microcontroller, supported by the ESP-IDF, facilitates the creation and management of these tasks through its APIs.

3.1 WORK FLOW OF THE PROPOSED METHOD

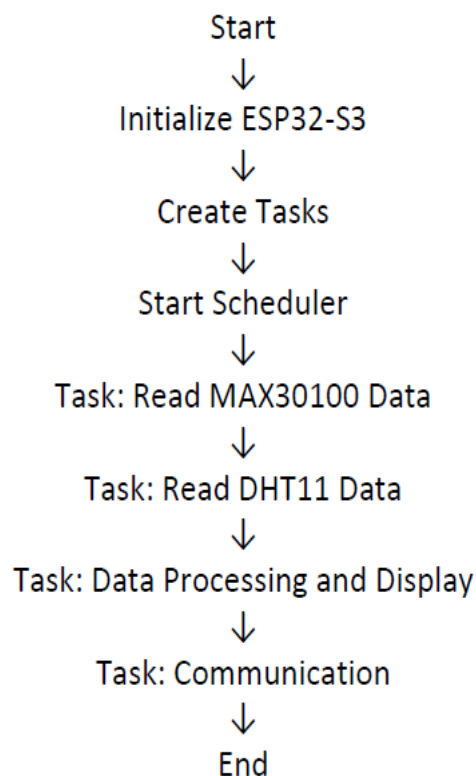


Figure 3.1: Flow chart of Task Synchronization

The process begins with the initialization of the ESP32-S3 microcontroller. This involves setting up the GPIO pins, initializing the I2C protocol for the MAX30100 pulse oximeter, and initializing the DHT11 temperature and humidity sensor. Additionally, FreeRTOS is initialized to manage the tasks.

Next, several tasks are created to handle different functions. A task is created for reading data from the MAX30100 sensor, another for reading data from the DHT11 sensor, a third task for processing and displaying the data, and an optional task for communication if needed.

Once the tasks are set up, the FreeRTOS scheduler is started, and it begins executing tasks based on their priority and current state. The task for reading data from the MAX30100 sensor reads SpO2 and BPM values using the I2C protocol and stores this data in shared variables. Mutexes or semaphores are used for synchronization to ensure data integrity, and once the data is read, the task signals the data processing task.

Similarly, the task for reading data from the DHT11 sensor reads temperature and humidity values and stores this data in shared variables. It also uses mutexes or semaphores for synchronization and signals the data processing task once the data is read.

The data processing and display task waits for the sensor data using synchronization mechanisms, processes the received data, and updates the display or logs the data. If the processed data needs to be sent to a server or an external device, this task signals the communication task.

Finally, the optional communication task sends the processed data to a server or external device. The process continues in a loop, with the scheduler managing the execution of tasks to ensure the patient health monitoring system operates efficiently and accurately.

3.2 Creation of Tasks Using Sensors

3.2.1. Task Definition: Each sensor is assigned a dedicated task to handle data reading and processing. For instance, the MAX30100 sensor task reads pulse and oxygen saturation levels, while the DHT11 task captures temperature and humidity data.

3.2.2. Task Creation: Using the `xTaskCreate` function from FreeRTOS, tasks are initialized with specified priorities, stack sizes, and handler functions. This allows for efficient concurrent execution of sensor readings.

3.3 Implementing Semaphores for Task Synchronization

To ensure reliable data collection and prevent data conflicts between tasks, semaphores are implemented as synchronization mechanisms.

3.3.1. Semaphore Definition: Semaphores act as signaling tools that control access to shared resources. In this project, semaphores are used to prevent multiple tasks from accessing sensor data simultaneously.

3.3.2. Implementation: The semaphore is created using the **xSemaphore** function. Each task requests the semaphore before accessing the shared resource, ensuring orderly execution. If a task is unable to obtain the semaphore, it enters a blocked state until the resource is available.

3.4 Commands:

The `idf.py` command is a command-line tool used to manage projects based on the Espressif IoT Development Framework (ESP-IDF). ESP-IDF is the official development framework for Espressif systems, specifically designed for developing applications for the ESP32 series of microcontrollers.

`idf.py` is essentially a helper script that provides a simplified interface to perform various development tasks related to ESP-IDF. It consolidates multiple build system commands and simplifies the workflow for developers. Here's a breakdown of what `idf.py` does and some of its

Common `idf.py` Commands:

3.4.1.`idf.py` `set-target`:

This command is used to set the target microcontroller for your project. For example, if you are using the ESP32-S3, you would run `idf.py set-target esp32s3`.

3.4.2.`idf.py` `menuconfig`:

Opens a terminal-based configuration menu where you can configure various settings for your project, such as pin assignments, library options, and other project-specific configurations.

3.4.3.`idf.py` `build`:

Compiles the source code of your project and generates the firmware binary files that can be flashed onto the ESP32 microcontroller.

3.4.4.idf.py flash:

Uploads the compiled firmware binary to the ESP32 microcontroller via a serial connection. This command typically also erases the previous firmware before flashing the new one.

3.4.5.idf.py monitor:

Opens a serial monitor to view debug output from the ESP32. This is useful for debugging and logging purposes, allowing you to see real-time output from the microcontroller.

3.4.6.idf.py clean:

Cleans the build directory by removing all compiled files. This is useful if you want to start the build process from scratch.

3.4.7.idf.py fullclean:

Removes all generated files, including configuration files. It's a more thorough cleanup than `idf.py clean` and is useful for troubleshooting build issues.

This chapter has detailed the implementation of task synchronization and CPU utilization for data acquisition in a health monitoring application using the **ESP32-S3** microcontroller. We outlined the workflow from the initialization of sensors to the creation of tasks for reading data and processing it efficiently. The use of semaphores was crucial in maintaining data integrity and preventing conflicts between concurrent tasks.

Additionally, the role of the **idf.py** command-line tool was emphasized, demonstrating how it streamlines the development process within the **ESP-IDF** framework.

As we move forward to the next chapter, we will evaluate the performance of the implemented system, analysing CPU utilization. This evaluation will provide insights into the effectiveness of the task synchronization strategies.

CHAPTER-4

Proposed Methodology for Task Synchronization

In real-time systems, executing multiple tasks simultaneously can lead to data corruption, lack of concurrency, and some tasks not being completed within the given time frame. To improve system reliability and performance, we developed a prototype using a few sensors and an ESP32-S3 controller.

To work with the ESP32-S3 controller, we installed the required ESP-IDF framework, which provides the necessary toolchains and compilers for the board. The ESP32-S3 was chosen for its dual-core capabilities, making it suitable for handling concurrent tasks. FreeRTOS is already ported on the board, facilitating task creation.

Initially, we connected sensors such as the Pulse Oximeter (MAX30100) and DHT11, and collected their outputs. Task creation was then performed using these sensors. We used the `xTaskCreate` command to create tasks, assigning priority, stack size, handlers, etc. After task creation, we observed how the tasks functioned under different priority and delay settings, noting that task overlapping occurred.

4.1 Communication Protocols being used:

The I2C (Inter-Integrated Circuit) protocol is a synchronous, multi-master, multi-slave communication standard that uses two lines: Serial Data Line (SDA) and Serial Clock Line (SCL). When connecting the MAX30100 sensor to the ESP32-S3, the SDA and SCL lines of the sensor are connected to the corresponding GPIO pins configured for I2C on the ESP32-S3. The MAX30100 communicates with the microcontroller via the I2C protocol, allowing the ESP32-S3 to read SpO2 and BPM data from the sensor. Initialization involves setting up the I2C driver on the ESP32-S3, configuring the GPIO pins for SDA and SCL, and setting the I2C frequency (commonly 100kHz or 400kHz). Data from the MAX30100 is read by sending commands over I2C and retrieving the data from specific registers on the sensor.

UART (Universal Asynchronous Receiver-Transmitter) is a simple, asynchronous serial communication protocol that uses separate lines for transmitting (TX) and receiving (RX) data. When connecting the DHT11 sensor to the ESP32-S3, the sensor's data pin is connected to a

GPIO pin on the ESP32-S3 that is configured for UART communication. The ESP32-S3 initializes the UART driver, specifying parameters such as baud rate (commonly 9600 bps), data bits, parity, and stop bits. Unlike I2C, UART does not require a clock line, making it simpler but less suitable for multi-device communication on the same bus. The ESP32-S3 sends a start signal to the DHT11 and then reads the temperature and humidity data sent back by the sensor over the UART line. This data is then processed and can be used for further applications, such as logging or display.

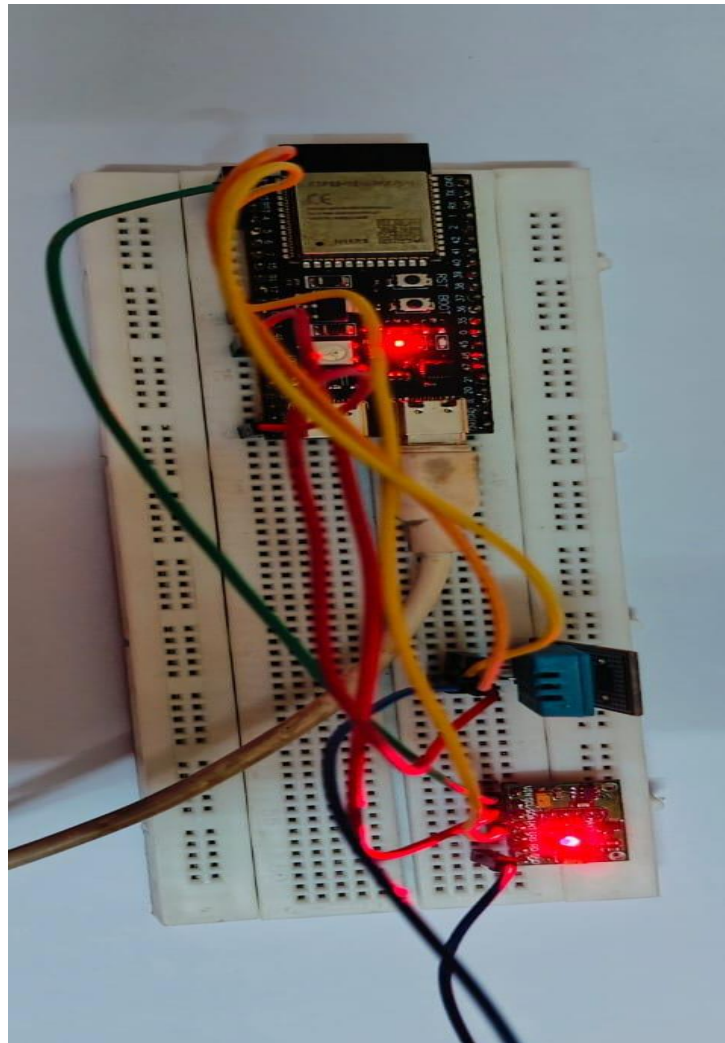


Figure 4.1: Proposed Design

4.2 Task Synchronization of Dummy Task

```
sensing : 1
processing: 0
main task : 1
sensing : 1
processing: 0
main task : 1
sensing : 1
processing: 0
main task : 1
sensing : 1
processing: 0
main task : 1
```

Figure 4.2:Output of Dummy Tasks

The use of FreeRTOS on an ESP32 to manage concurrent tasks with synchronization using a binary semaphore. Two tasks, SensingTask and ProcessingTask, are created to increment and decrement a shared variable countval respectively. Each task first takes the semaphore using xSemaphoreTake to ensure exclusive access to countval, modifies the variable, prints its value, and then releases the semaphore using xSemaphoreGive. These tasks run periodically every 5 seconds. The app_main function initializes the semaphore and creates the two tasks with equal priority (priority 5). Additionally, the main task runs in an infinite loop, periodically printing its own priority every 5 seconds, showcasing concurrent execution with synchronization to avoid data races on the shared variable count value.

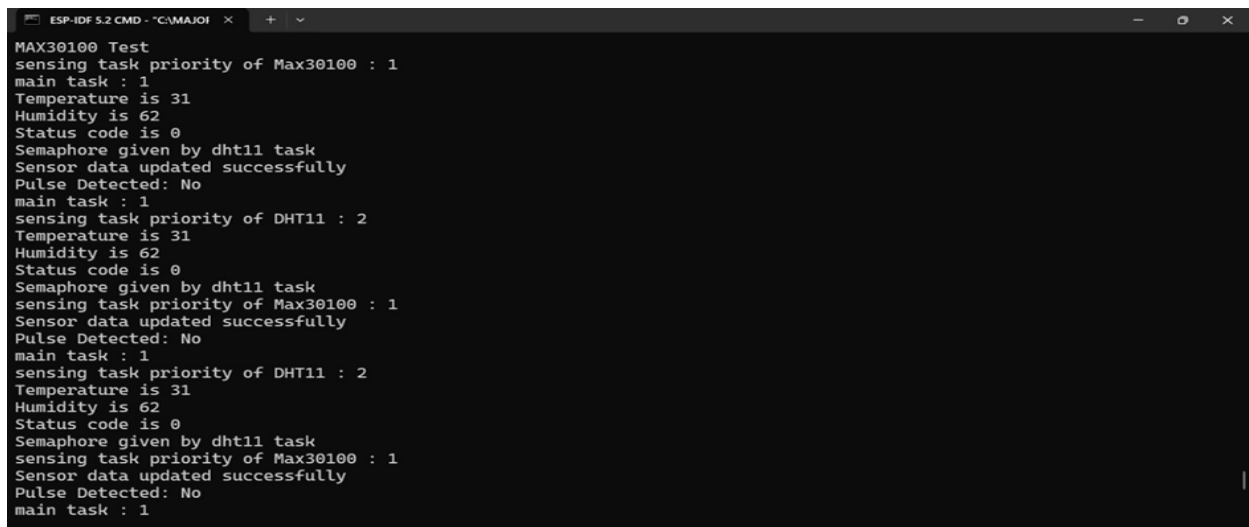
4.3 Task Synchronization of proposed design

The program starts by initializing the DHT11 sensor on GPIO pin 4 (GPIO_NUM_4) using DHT11_init(). It then initializes the I2C communication (I2C_NUM_0) with the specified SDA (I2C_SDA) and SCL (I2C_SCL) pins and frequency (I2C_FRQ). The MAX30100 sensor is initialized using max30100_init(), configuring it with default settings such as operating mode, sampling rate, LED pulse width, and current settings.

A binary semaphore (xSemaphore) is created using xSemaphoreCreateBinary(). Semaphores are synchronization primitives used to control access to shared resources, in this case, the sensor data. If the semaphore creation fails (xSemaphore == NULL), an error message is printed, and the program exits.

Task synchronization is the coordination of multiple tasks to ensure they operate correctly when sharing resources or communicating with each other. In this program, semaphore (xSemaphore) is used for synchronization. Semaphores allow tasks to control access to shared resources in a way that prevents conflicts and ensures data integrity.

The semaphore (xSemaphore) is crucial for ensuring that the get_bpm task waits until new data is available from the dht11 task before attempting to read from the MAX30100 sensor.



```
ESP-IDF 5.2 CMD - C:\MAJOI
MAX30100 Test
sensing task priority of Max30100 : 1
main task : 1
Temperature is 31
Humidity is 62
Status code is 0
Semaphore given by dht11 task
Sensor data updated successfully
Pulse Detected: No
main task : 1
sensing task priority of DHT11 : 2
Temperature is 31
Humidity is 62
Status code is 0
Semaphore given by dht11 task
sensing task priority of Max30100 : 1
Sensor data updated successfully
Pulse Detected: No
main task : 1
sensing task priority of DHT11 : 2
Temperature is 31
Humidity is 62
Status code is 0
Semaphore given by dht11 task
sensing task priority of Max30100 : 1
Sensor data updated successfully
Pulse Detected: No
main task : 1
```

Figure 4.3: Output of Task Synchronization after connecting sensors to esp32s3

Without the semaphore:

The get_bpm task might read outdated or inconsistent data from the MAX30100 sensor if it doesn't wait for the dht11 task to complete. Both tasks might also attempt to access shared resources simultaneously, leading to race conditions and potentially corrupting data.

CHAPTER-5

Sensor Reading With Application Of Semaphore And Observing CPU Utilization Time

Case-1 :

Priority

- MAX30100 =1
- DHT11=1
- Main=1

Delay

- MAX30100 =10 sec
- DHT11=5 sec
- Main=1sec

Task	Cpu utilization time without Semaphore(ms)	Cpu utilization time with Semaphore (ms)	Cpu utilization time with Semaphore singlecore(ms)	Cpu utilization time with Semaphore and dualcore(ms)
Max 30100	9143	38117	15140	30935
DHT11	25301	30335	42323	26715
Max 30100	9143	38117	15140	30935
DHT11	24240	24407	24493	24502
Max 30100	1172	3562	-	-
DHT11	24262	24605	-	-

Case-2 :

Priority

- MAX30100 =1
- DHT11=1
- Main=1

Delay

- MAX30100 =1sec
- DHT11=1sec
- Main=1sec

Task	Cpu utilization time without Semaphore(ms)	Cpu utilization time with Semaphore (ms)	Cpu utilization time with Semaphore singlecore(ms)	Cpu utilization time with Semaphore and dualcore(ms)
Max 30100	9143	38120	15411	3526
DHT11	25313	30481	42361	10676
Max 30100	1164	5677	3526	3093
DHT11	927	1091	1095	2690
Max 30100	1158	5678	3510	3527
DHT11	24230	1190	24752	24501

Case-3 :

Priority

- MAX30100 =2
- DHT11=3
- Main=1

Delay

- MAX30100 =10sec
- DHT11=5sec
- Main=1sec

Task	Cpu utilization time without Semaphore(ms)	Cpu utilization time with Semaphore (ms)	Cpu utilization time with Semaphore singlecore(ms)	Cpu utilization time with Semaphore and dualcore(ms)
Max 30100	1505	0	11607	0
DHT11	24279	36151	24861	24915
Max 30100	1505	36736	11607	41382
DHT11	24518	24502	24590	24586
Max 30100	1167	3527	10056	36511
DHT11	24685	24523	24790	24495

Case-4 :

Priority

- MAX30100 =2
- DHT11=1
- Main=3

Delay

- MAX30100 =1sec
- DHT11=1sec
- Main=1sec

Task	Cpu utilization time without Semaphore(ms)	Cpu utilization time with Semaphore (ms)	Cpu utilization time with Semaphore singlecore(ms)	Cpu utilization time with Semaphore and dualcore(ms)
Max 30100	1505	16744	11498	9713
DHT11	24406	1095	24960	24519
Max 30100	1153	5680	10362	1886
DHT11	24327	1094	24502	1091
Max 30100	1156	5679	10387	1890
DHT11	24298	1091	24465	1092

The tables presented provide a detailed analysis of CPU utilization times for tasks involving MAX30100 and DHT11 sensors under various priority and delay settings, both with and without semaphores, and across single-core and dual-core setups. Each case highlights the importance of task synchronization and how semaphores affect the performance and efficiency of these tasks.

Task synchronization using semaphores is crucial for managing multiple tasks with the same or varying priorities. Semaphores help ensure that tasks do not overlap or interfere with each other, leading to more predictable and stable system behavior. Without synchronization, tasks can experience significant variability in execution times, resulting in inefficient CPU usage and potential system instability. By using semaphores, tasks can be coordinated to run more smoothly, although this comes with some overhead. In dual-core setups, the benefits of synchronization are maximized, as the workload is distributed more evenly across multiple cores, significantly improving system efficiency and performance.

In **Case-1**, with equal priority for all tasks and significant delays (10 seconds for MAX30100, 5 seconds for DHT11, and 1 second for Main), the CPU utilization without semaphores shows high variability, particularly for DHT11, which ranges from 24240 ms to 25301 ms. With semaphores, there is a notable increase in CPU utilization, but it also brings more consistency, especially in dual-core setups where MAX30100 and DHT11 tasks have significantly reduced CPU times. This highlights the semaphores' role in coordinating task execution, albeit with some overhead.

Case-2 explores a scenario where all tasks have equal priority, and the delays are much shorter (1 second each for MAX30100, DHT11, and Main). Without semaphores, CPU utilization is inconsistent, but with semaphores, it becomes more uniform. The dual-core setup in this case exhibits the best performance, with substantial reductions in CPU times for both sensors, indicating efficient task distribution and synchronization. This case demonstrates the benefits of shorter delays and the effectiveness of semaphores in improving task coordination and performance.

In **Case-3**, with different priorities (MAX30100=2, DHT11=3, Main=1) and longer delays (10 seconds for MAX30100, 5 seconds for DHT11, 1 second for Main), the tasks show significant CPU utilization variability without semaphores. With semaphores, there is a marked improvement in consistency, especially in single-core setups. However, the dual-core setup

provides the best performance, indicating that priority settings and synchronization mechanisms effectively manage task execution, reducing conflicts and improving overall system efficiency.

Case-4 examines a scenario where MAX30100 has medium priority (2), DHT11 has high priority (1), and Main has low priority (3), with short delays (1 second each). Without semaphores, CPU utilization is again variable, but with semaphores, it becomes more consistent. The dual-core setup outperforms others, showcasing reduced and stable CPU times for both sensors. This case highlights the importance of assigning appropriate priorities and using synchronization to optimize task execution and system performance.

Among the cases, **Case-2** stands out as the best due to its efficient and consistent CPU utilization times, particularly in the dual-core setup. This setup achieves the lowest CPU times (MAX30100 at 3526 ms and DHT11 at 10676 ms), demonstrating effective task synchronization and distribution. The shorter delays and equal priorities ensure that tasks are managed well without unnecessary conflicts or delays.

In real-life scenarios, such as in embedded systems for healthcare monitoring, efficient and consistent task execution is critical. For instance, in a wearable health monitor that uses sensors like MAX30100 for heart rate and DHT11 for temperature and humidity, synchronized task execution ensures reliable data collection and processing. Without proper synchronization, the device may fail to provide accurate real-time data due to conflicting sensor reads and processing delays. Implementing semaphores ensures that each sensor's data is collected and processed efficiently, enhancing the device's reliability and performance. Utilizing a dual-core processor further optimizes this by distributing the tasks, ensuring the device can handle real-time data processing without compromising on performance or accuracy.

CHAPTER-6

RESULTS

Our findings show that the use of semaphores will reduces CPU utilization time in dual-core setups. For instance, in the absence of semaphores, tasks like sensor readings often conflicted, leading to increased CPU time.

Case-1:

```
ESP-IDF 5.2 CMD - "C:\MAJOF" x + v
W (267) spi_flash: Detected size(16384k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
W (281) i2c: This driver is an old driver, please migrate your application code to adapt 'driver/i2c_master.h'
I (291) sleep: Configure to isolate all GPIO pins in sleep state
I (298) sleep: Enable automatic switching of GPIO sleep configuration
I (306) main_task: Started on CPU0
I (316) main_task: Calling app_main()
MAX30100 Test
sensing task: 2
main task: 1
get_bpm CPU utilization time: 0
dht11 CPU utilization time: 0
Sensor data updated successfully
Pulse Detected: No
Temperature is 28
Humidity is 72
Status code is 0
main task: 1
get_bpm CPU utilization time: 3070
dht11 CPU utilization time: 24646
Sensor data updated successfully
Pulse Detected: No
main task: 1
get_bpm CPU utilization time: 1075
dht11 CPU utilization time: 24646
Sensor data updated successfully
Pulse Detected: No
main task: 1
get_bpm CPU utilization time: 1034
dht11 CPU utilization time: 24646
Sensor data updated successfully
Pulse Detected: No
main task: 1
```

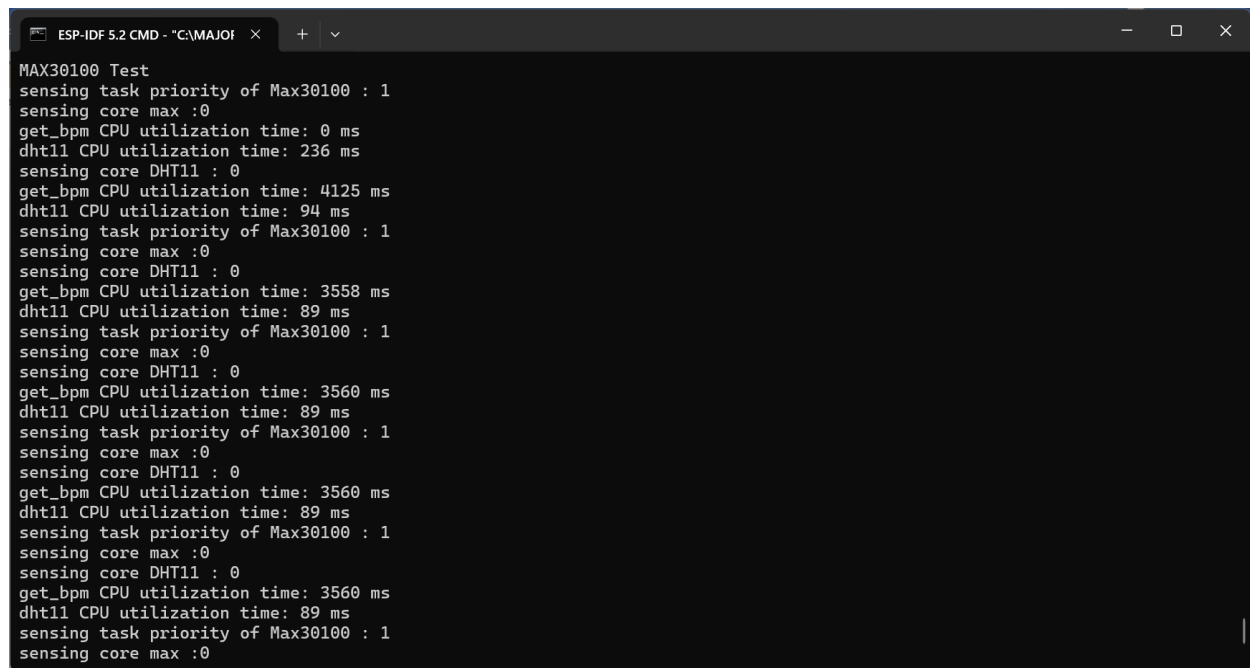
Figure 6.1: CPU Utilization Time Of Sensor

Case-2:

```
sensing task : 2
MAX30100 Test
Temperature is 26
Humidity is 39
Status code is 0
Semaphore given by dht11 task
Semaphore taken by get_bpm task
main task : 1
main task : 1
main task : 1
main task : 1
main task : 1
sensing task : 2
Temperature is 26
Humidity is 39
Status code is 0
Semaphore given by dht11 task
Semaphore taken by get_bpm task
main task : 1
main task : 1
main task : 1
main task : 1
main task : 1
sensing task : 2
Temperature is 26
Humidity is 39
Status code is 0
Semaphore given by dht11 task
Semaphore taken by get_bpm task
main task : 1
```

Figure 6.2: CPU Utilization Time Of Sensor with Semaphore

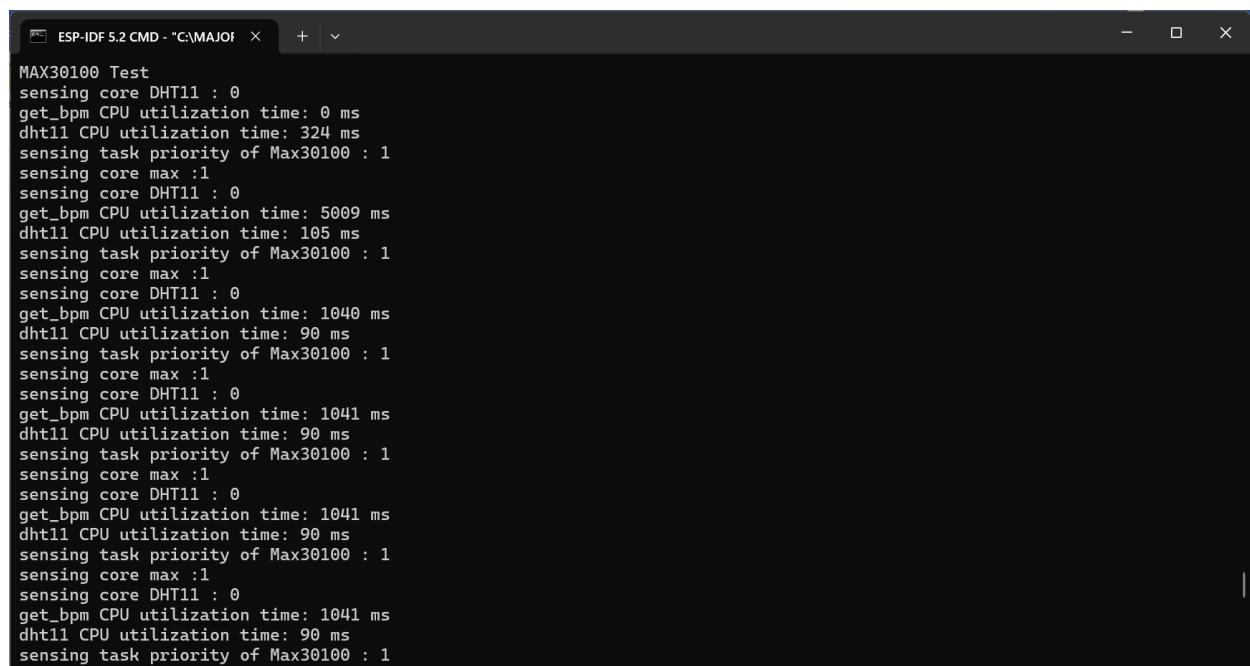
Case-3:



```
ESP-IDF 5.2 CMD - "C:\MAJOF" x + v
MAX30100 Test
sensing task priority of Max30100 : 1
sensing core max :0
get_bpm CPU utilization time: 0 ms
dht11 CPU utilization time: 236 ms
sensing core DHT11 : 0
get_bpm CPU utilization time: 4125 ms
dht11 CPU utilization time: 94 ms
sensing task priority of Max30100 : 1
sensing core max :0
sensing core DHT11 : 0
get_bpm CPU utilization time: 3558 ms
dht11 CPU utilization time: 89 ms
sensing task priority of Max30100 : 1
sensing core max :0
sensing core DHT11 : 0
get_bpm CPU utilization time: 3560 ms
dht11 CPU utilization time: 89 ms
sensing task priority of Max30100 : 1
sensing core max :0
sensing core DHT11 : 0
get_bpm CPU utilization time: 3560 ms
dht11 CPU utilization time: 89 ms
sensing task priority of Max30100 : 1
sensing core max :0
sensing core DHT11 : 0
get_bpm CPU utilization time: 3560 ms
dht11 CPU utilization time: 89 ms
sensing task priority of Max30100 : 1
sensing core max :0
```

Figure 6.3: CPU Utilization Time Of Sensor with Semaphore in single core

Case-4:



```
ESP-IDF 5.2 CMD - "C:\MAJOF" x + v
MAX30100 Test
sensing core DHT11 : 0
get_bpm CPU utilization time: 0 ms
dht11 CPU utilization time: 324 ms
sensing task priority of Max30100 : 1
sensing core max :1
sensing core DHT11 : 0
get_bpm CPU utilization time: 5009 ms
dht11 CPU utilization time: 105 ms
sensing task priority of Max30100 : 1
sensing core max :1
sensing core DHT11 : 0
get_bpm CPU utilization time: 1040 ms
dht11 CPU utilization time: 90 ms
sensing task priority of Max30100 : 1
sensing core max :1
sensing core DHT11 : 0
get_bpm CPU utilization time: 1041 ms
dht11 CPU utilization time: 90 ms
sensing task priority of Max30100 : 1
sensing core max :1
sensing core DHT11 : 0
get_bpm CPU utilization time: 1041 ms
dht11 CPU utilization time: 90 ms
sensing task priority of Max30100 : 1
sensing core max :1
sensing core DHT11 : 0
get_bpm CPU utilization time: 1041 ms
dht11 CPU utilization time: 90 ms
sensing task priority of Max30100 : 1
```

Figure 6.4: CPU Utilization Time Of Sensor with Semaphore in dual core

CHAPTER-7

CONCLUSION AND FUTURE SCOPE

This project focused on the **implementation and evaluation of task synchronization** and **CPU utilization analysis** for sensor readings using the **ESP32-S3 microcontroller**. We designed a system that acquires data from the **MAX30100 pulse oximeter** and **DHT11 temperature and humidity sensor** in real-time, employing **FreeRTOS** to manage tasks. Our methodology utilized **semaphores** to ensure efficient task synchronization, reducing conflicts in accessing shared resources and minimizing CPU utilization.

The findings demonstrated that **semaphores significantly reduced CPU utilization time**, particularly in **dual-core setups**, by preventing task conflicts. Without semaphores, tasks such as sensor readings competed for resources, increasing CPU time and system inefficiency. In contrast, semaphore implementation enabled smooth task execution by controlling access to shared data, thus optimizing system performance.

Using the **ESP-IDF framework** with the **idf.py** toolset facilitated efficient development, compilation, and debugging of the system. Overall, the integration of semaphores, task scheduling, and robust development tools contributed to a reliable and efficient system capable of real-time sensor data acquisition and processing.

Future Scope

The system can be expanded by integrating additional sensors to monitor more parameters, allowing for a wider range of applications. Exploring **scalability** will enable the system to handle larger datasets and more complex sensor networks without sacrificing performance. Future work could focus on evaluating **multi-core performance** with more advanced tasks, ensuring the system remains efficient even with increasing sensor load.

Another direction for future work is optimizing the system's **energy efficiency** to make it more suitable for **battery-powered** or portable applications. Implementing **low-power modes** and refining power management techniques could extend the system's operational life. Additionally, incorporating **wireless data transmission** capabilities such as real-time data sharing with cloud-based platforms will enable **remote monitoring**, which could enhance its application in healthcare, environmental monitoring, and other real-time monitoring systems.

APPENDIX

I. Task Synchronization:

Task synchronization in FreeRTOS is crucial for ensuring that tasks coordinate effectively, especially when they share resources or need to communicate. Key mechanisms for synchronization include:

1. Semaphores:

- Binary Semaphores: Used to signal between tasks or between an interrupt and a task. It acts as a simple flag to indicate the availability of a resource or the occurrence of an event.
- Counting Semaphores: Used to manage a resource pool where multiple instances of a resource are available.

2. Mutexes (Mutual Exclusions):

- Used to protect shared resources, ensuring that only one task accesses a resource at a time.
- FreeRTOS mutexes also include priority inheritance to prevent priority inversion, where a higher-priority task is waiting for a resource held by a lower-priority task.

3. Queues:

- Used for inter-task communication, allowing tasks to send and receive messages in a thread-safe manner.
- Queues can store data items (e.g., structures, integers) and ensure that data is transferred safely between tasks.

4. Event Groups:

- Used for task synchronization, allowing a task to wait for multiple events. Each event is represented by a bit in an event group, and tasks can wait for one or more bits to be set.

5. Task Notifications:

- Lightweight and efficient way to send data to tasks and unblock tasks. Each task has a 32-bit value that can be used for simple event signaling or data transfer.

Practical Application:

In the context of a wearable health monitoring device, task scheduling and synchronization ensure that various components work together seamlessly:

1. Data Acquisition Task: Periodically collects data from sensors. Uses a timer to trigger data collection at fixed intervals.

2. Data Processing Task: Processes the collected data to compute health metrics. This task might wait on a semaphore or event group to start processing when new data is available.

3. Communication Task: Sends processed data to an external device (e.g., smartphone, cloud). Uses a queue to receive data from the processing task and then handles communication.

4. Alert Task: Monitors processed data for critical conditions and triggers alerts. Uses task notifications to get data from the processing task and generate alerts if necessary.

By leveraging FreeRTOS for task scheduling and synchronization, the wearable health monitoring device can achieve reliable, real-time performance, ensuring accurate and timely monitoring of vital signs. This modular approach simplifies development and enhances the maintainability and scalability of the system.

II. HARDWARE COMPONENTS:

- 1.ESP32 microcontroller (Wi-Fi module/ web server)
- 2.DS18B20 sensor (Body temperature sensor)
- 3.DHT11 sensor (Room temperature & humidity sensor)
- 4.MAX30100 sensor (Pulse oximeter & heart rate sensor)
- 5.Resistor, jumper Wirer & Breadboard

ESP32 is a low-cost System on Chip (SoC) Microcontroller from Espressif Systems, the developers of the famous ESP8266 SoC. It is a successor to ESP8266 SoC and comes in both single-core and dual-core variations of the Tensilica's 32-bit Xtensa LX6 Microprocessor with integrated Wi-Fi and Bluetooth. The good thing about ESP32, like ESP8266 is its integrated RF components like Power Amplifier, Low-Noise Receive Amplifier, Antenna Switch, Filters and RF Balun. This makes designing hardware around ESP32 very easy as you require very few external components. The ESP32 microcontroller is shown in below figure-

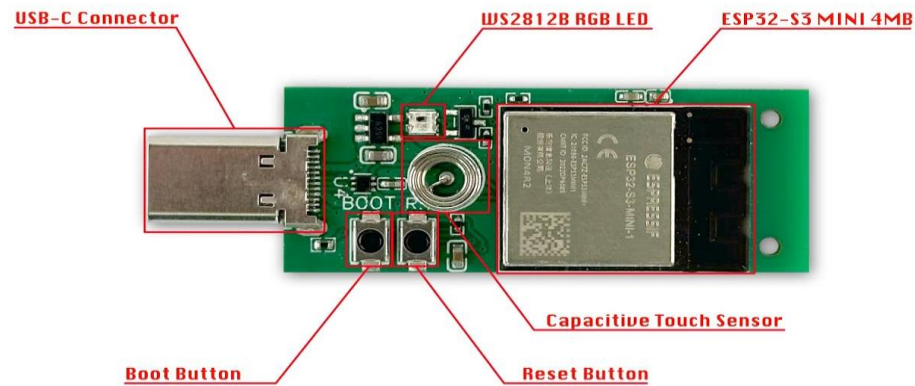


Figure a: Internal Structure of ESP32S3

The ESP32 Board consists of the following:

- 1.ESP-WROOM-32 Module
- 2.Two rows of IO Pins (with 15 pins on each side)
- 3.CP2012 USB – UART Bridge IC
- 4.micro–USB Connector (for power and programming)
- 5.AMS1117 3.3V Regulator IC
- 6.Enable Button (for Reset)
- 7.Boot Button (for flashing)
- 8.Power LED (Red)
- 9.User LED (Blue – connected to GPIO2)

An interesting point about the USB-to-UART IC is that its DTR and RTS pins are used to automatically set the ESP32 in to programming mode (whenever required) and also rest the board after programming.

Pin out of ESP32 Board:

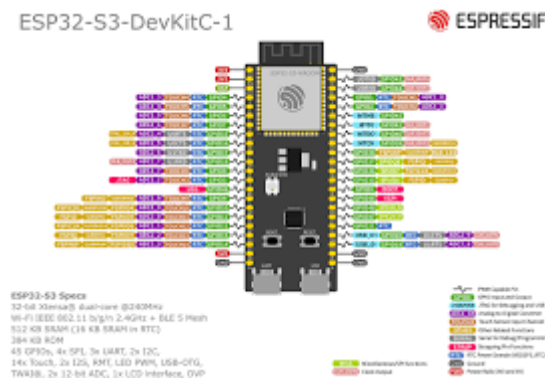


Figure b: pin out diagram of the ESP32 Development Board.

This pin out is for the 30 – pin version of the ESP Board. In the pin out tutorial, I will explain the pin out of both the 30 – pin as well as the 36 – pin version of the ESP Boards.

Interfacing MAX30100 Pulse Ox meter and Heart Rate Sensor:

The module features the MAX30100 – a modern, integrated pulse ox meter and heart rate sensor IC, from Analog Devices. It combines two LEDs, a photo detector, optimized optics, and low-noise analog signal processing to detect pulse oximetry (SpO₂) and heart rate (HR) signals.

On the right, the MAX30100 has two LEDs – a RED and an IR LED. And on the left is a very sensitive photodetector. The idea is that you shine a single LED at a time, detecting the amount of light shining back at the detector, and, based on the signature, you can measure blood oxygen level and heart rate.

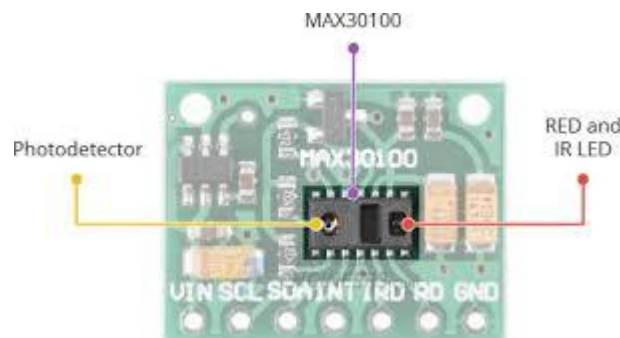


Figure c: Max30100 Sensor

Power Requirement:

The MAX30100 chip requires two different supply voltages: 1.8V for the IC and 3.3V for the RED and IR LEDs. So the module comes with 3.3V and 1.8V regulators. This allows you to connect the module to any microcontroller with 5V, 3.3V, even 1.8V level I/O. One of the most important features of the MAX30100 is its low power consumption.



Figure d: Max30100 Sensor based on power Requirements

than $600\mu\text{A}$ during measurement. Also, it is possible to put the MAX30100 in standby mode, where it consumes only $0.7\mu\text{A}$. This low power consumption allows implementation in battery powered devices such as handsets, wearables or smart watches.

On-Chip Temperature Sensor:

The MAX30100 has an on-chip temperature sensor that can be used to compensate for the changes in the environment and to calibrate the measurements.

This is a reasonably precise temperature sensor that measures the ‘die temperature’ in the range of -40°C to $+85^{\circ}\text{C}$ with an accuracy of $\pm 1^{\circ}\text{C}$.

I2C Interface: The module uses a simple two-wire I2C interface for communication with the microcontroller. It has a fixed I2C address: $0xAE_{\text{HEX}}$ (for write operation) and $0xAF_{\text{HEX}}$ (for read operation).

FIFO Buffer:

The MAX30100 embeds a FIFO buffer for storing data samples. The FIFO has a 16-sample memory bank, which means it can hold up to 16 SpO₂ and heart rate samples. The FIFO buffer can offload the microcontroller from reading each new data sample from the sensor, thereby saving system power.

Interrupts:

The MAX30100 can be programmed to generate an interrupt, allowing the host microcontroller to perform other tasks while the data is collected by the sensor. The interrupt can be enabled for 5 different sources:

- 1.Power Ready: triggers on power-up or after a brownout condition.
- 2.SpO2 Data Ready: triggers after every SpO2 data sample is collected
- 3.Heart Rate Data Ready: triggers after every heart rate data sample is collected.
- 4.Temperature Ready: triggers when an internal die temperature conversion is finished.
- 5.FIFO Almost Full: triggers when the FIFO becomes full and future data is about to lost.

The INT line is an open-drain, so it is pulled HIGH by the onboard resistor. When an interrupt occurs the INT pin goes LOW and stays LOW until the interrupt is cleared.

Here are the technical specifications:

Power supply :3.3V to 5.5V

Current draw: $\sim 0.7\mu\text{A}$

$\sim 0.7\mu\text{A}$ (during standby mode

Red LED Wavelength :660nm

IR LED Wavelength :880nm

Temperature Range: -40°C to $+85^{\circ}\text{C}$

Temperature Accuracy: $\pm 1^{\circ}\text{C}$

Working principle of MAX30100 Pulse Oximeter and Heart Rate Sensor:

The MAX30100 or any optical pulse oximeter and heart-rate sensor for that matter, consists of a pair of high-intensity LEDs (RED and IR both of different wavelengths) and a photo detector. The wavelengths of these LEDs are 660nm and 880nm respectively. The MAX30100 works by shining both lights onto the finger or earlobe (or essentially anywhere where the skin photo detector. This method of pulse detection through light is called Photoplethysmogram.

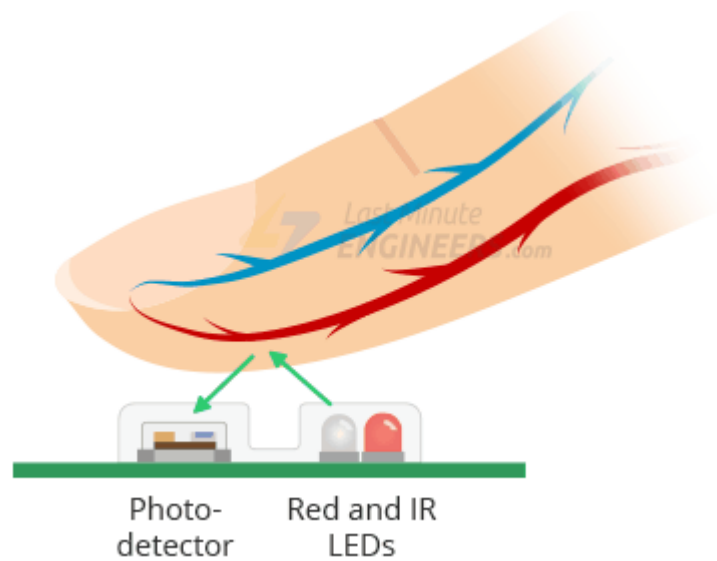


Figure e: Max30100 Working Phenomenon

The working of MAX30100 can be divided into two parts: Heart Rate Measurement and Pulse Oximetry (measuring the oxygen level of the blood).

Heart Rate Measurement:

The oxygenated hemoglobin (HbO_2) in the arterial blood has the characteristic of absorbing IR light. The redder the blood (the higher the hemoglobin), the more IR light is absorbed. As the blood is pumped through the finger with each heartbeat, the amount of reflected light changes, creating a changing waveform at the output of the photodetector. As you continue to shine light and take photo detector readings, you quickly start to get a heart-beat (HR) pulse reading.

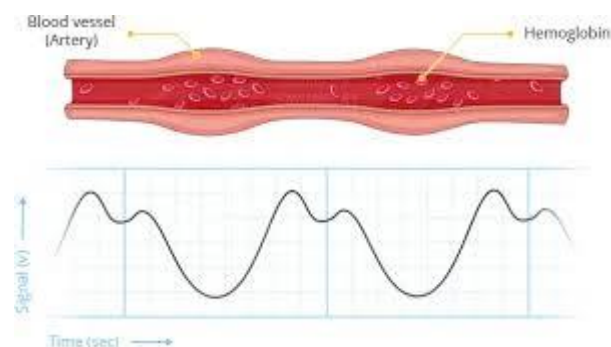


Figure f: Heart Rate Measurement

MAX30100 Module Pinout:

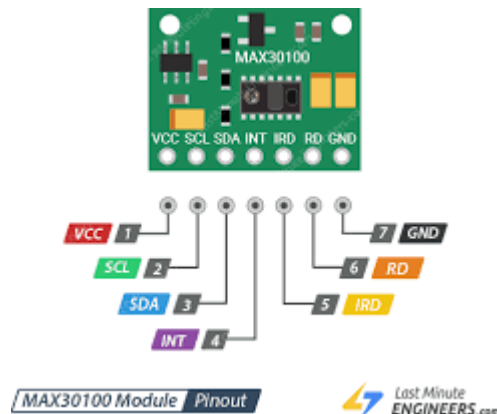


Figure g: Max30100 Sensor PIN Diagram

The MAX30100 module brings out the following connections:

- VIN- is the power pin. You can connect it to 3.3V or 5V output from your Arduino.
- SCL is the I2C clock pin, connect to your Arduino's I2C clock line.
- SDA is the I2C data pin, connect to your Arduino's I2C data line.
- INT-The MAX30100 can be programmed to generate an interrupt for each pulse. This line is open-drain, so it is pulled HIGH by the onboard resistor. When an interrupt occurs the INT pin goes LOW and stays LOW until the interrupt is cleared.
 - IRD- The MAX30100 integrates an LED driver to drive LED pulses for SpO2 and HR measurements. Use this if you want to drive the IR LED yourself, otherwise leave it unconnected.
- RD pin is similar to the IRD pin, but is used to drive the Red LED. If you don't want to drive the red LED yourself,
- GND is the ground.

CIRCUIT DIAGRAM:

Circuit diagram of sensors connected with ESP32 Web Server. All the sensors can work at 3.3V VCC. So connect their VCC to 3.3V Power Supply. Connect the GND to GND. MAX30100 is an I2C Sensor, so connect its SDA & SCL to GPIO21 & GPIO22. Connect its INT pin to GPIO19 of ESP32. The output pin of DHT11 is connected to GPIO18 of ESP32. Similarly, the output pin of DS18B20 is connected to GPIO5 of ESP32. A 4.7K pull-up resistor is connected between output pin & VCC pin of DS18B20.

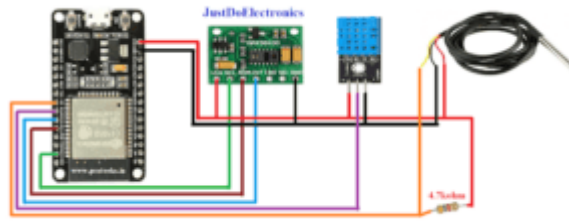


Figure h: Connections of Proposed design

https://www.researchgate.net/publication/372268729_IMPLEMENTATION_OF_IOT_BASED_PATIENT_HEALTH_MONITORING_SYSTEM_USING_ESP32_WEB_SERVER

III. Commands:

xTaskCreate

This function creates a new task and adds it to the list of tasks that are ready to run.

Syntax:

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode, const char * const pcName,
configSTACK_DEPTH_TYPE usStackDepth, void *pvParameters,
UBaseType_t uxPriority, TaskHandle_t *pxCreatedTask);
```

Parameters:

pvTaskCode: Pointer to the task function.

pcName: Name of the task.

usStackDepth: Stack size allocated for the task.

pvParameters: Parameters passed to the task.

uxPriority: Priority of the task.

pxCreatedTask: Handle to the created task.

vTaskDelete

Deletes a task and removes it from the scheduling list.

Syntax:

```
void vTaskDelete(TaskHandle_t xTask);
```

Parameters:

xTask: Handle of the task to delete. If NULL, the calling task is deleted.

vTaskDelay

Delays a task for a specified number of tick periods.

Syntax:

```
void vTaskDelay(const TickType_t xTicksToDelay);
```

Parameters:

xTicksToDelay: Number of tick periods to delay the task.

vTaskDelayUntil

Delays a task until a specified time.

Syntax:

```
void vTaskDelayUntil(TickType_t *pxPreviousWakeTime, const TickType_t xTimeIncrement);
```

Parameters:

pxPreviousWakeTime: Pointer to a variable that holds the time at which the task was last unblocked.

xTimeIncrement: Time interval at which the task should be unblocked.

vTaskPrioritySet

Changes the priority of a task.

Syntax:

```
void vTaskPrioritySet(TaskHandle_t xTask, UBaseType_t uxNewPriority);
```

Parameters:

xTask: Handle of the task whose priority is to be changed.

uxNewPriority: New priority to be assigned to the task.

Task Synchronization

xSemaphoreCreateMutex

Creates a mutex semaphore.

Syntax:

```
SemaphoreHandle_t xSemaphoreCreateMutex(void);
```

xSemaphoreTake

Takes a semaphore.

Syntax:

```
BaseType_t xSemaphoreTake(SemaphoreHandle_t xSemaphore, TickType_t xTicksToWait);
```

Parameters:

xSemaphore: Handle of the semaphore.

xTicksToWait: Number of tick periods to wait for the semaphore.

xSemaphoreGive

Gives a semaphore.

Syntax:

```
BaseType_t xSemaphoreGive(SemaphoreHandle_t xSemaphore);
```

Parameters:

xSemaphore: Handle of the semaphore.

xQueueCreate

Creates a queue.

Syntax:

```
QueueHandle_t xQueueCreate(UBaseType_t uxQueueLength, UBaseType_t uxItemSize);
```

Parameters:

uxQueueLength: Number of items the queue can hold.

uxItemSize: Size of each item in the queue.

xQueueSend

Sends an item to a queue.

Syntax:

```
BaseType_t xQueueSend(QueueHandle_t xQueue, const void *pvItemToQueue, TickType_t xTicksToWait);
```

Parameters:

xQueue: Handle of the queue.

pvItemToQueue: Pointer to the item to send.

xTicksToWait: Number of tick periods to wait for space in the queue.

xQueueReceive

Receives an item from a queue.

Syntax:

```
 BaseType_t xQueueReceive( QueueHandle_t xQueue, void *pvBuffer, TickType_t  
 xTicksToWait);
```

Parameters:

xQueue: Handle of the queue.

pvBuffer: Pointer to the buffer where the received item will be stored.

xTicksToWait: Number of tick periods to wait for an item.

IV. Installation process of esp-idf in windows including drivers

1. Installation process of esp-idf in windows including drivers

- Search for esp-idf in the chrome
- Go to the link
<https://docs.espressif.com/projects/esp-idf/en/v5.2.2/esp32s3/get-started/index.html>
- Select the required board and the version by default it will have esp32 according to our project we need esp32s3



Figure i:esp-idf

- Go to manual installation and in that go for window installer

Manual Installation

For the manual procedure, please select according to your operating system.

- [Windows Installer](#)
- [Linux and macOS](#)

- Install required tool chain

ESP-IDF Tools Installer

The easiest way to install ESP-IDF's prerequisites is to download one of ESP-IDF Tools Installers.



[Windows Installer Download](#)

ESP-IDF Windows Installer Download

Open Source IoT Development Framework for ESP32



Universal Online Installer 2.26
Windows 10, 11
Size: 4.24 MB



Espressif-IDE 3.0.0 with ESP-IDF
v5.2.2 - Offline Installer
Windows 10, 11
Size: 1.93 GB

- Follow the guide provided in the link then the command prompt will pop up Which means successful installation of esp-idf in window OS is done.

```
ESP-IDF 5.2 CMD - "C:\MAJOR" X + v
Setting PYTHONUSERSITE, was not set
Using Python in C:\MAJOR\esp\Espressif\python_env\idf5.2_py3.11_env\Scripts\
Python 3.11.2
Using Git in C:\MAJOR\esp\Espressif\tools\idf-git\2.43.0\cmd\
git version 2.43.0.windows.1
Checking Python compatibility
Setting IDF_PATH: C:\MAJOR\esp\Espressif\frameworks\esp-idf-v5.2.1

Adding ESP-IDF tools to PATH...
C:\MAJOR\esp\Espressif\tools\xtensa-esp-elf-gdb\12.1.20231023\xtensa-esp-elf-gdb\bin
C:\MAJOR\esp\Espressif\tools\riscv32-esp-elf-gdb\12.1.20231023\riscv32-esp-elf-gdb\bin
C:\MAJOR\esp\Espressif\tools\xtensa-esp-elf\esp-13.2.0_20230920\xtensa-esp-elf\bin
C:\MAJOR\esp\Espressif\tools\riscv32-esp-elf\esp-13.2.0_20230920\riscv32-esp-elf\bin
C:\MAJOR\esp\Espressif\tools\esp32ulp-elf\2.35.20220830\esp32ulp-elf\bin
C:\MAJOR\esp\Espressif\tools\cmake\3.24.0\bin
C:\MAJOR\esp\Espressif\tools\openocd-esp32\v0.12.0-esp32-20230921\openocd-esp32\bin
C:\MAJOR\esp\Espressif\tools\ninja\1.11.1\
C:\MAJOR\esp\Espressif\tools\idf-exe\1.0.3\
C:\MAJOR\esp\Espressif\tools\ccache\4.8\ccache-4.8-windows-x86_64
C:\MAJOR\esp\Espressif\tools\dfu-util\0.11\dfu-util-0.11-win64
C:\MAJOR\esp\Espressif\frameworks\esp-idf-v5.2.1\tools

Checking if Python packages are up to date...
Constraint file: C:\MAJOR\esp\Espressif\esp-idf.constraints.v5.2.txt
Requirement files:
- C:\MAJOR\esp\Espressif\frameworks\esp-idf-v5.2.1\tools\requirements\requirements.core.txt
Python being checked: C:\MAJOR\esp\Espressif\python_env\idf5.2_py3.11_env\Scripts\python.exe
Python requirements are satisfied.

Detected installed tools that are not currently used by active ESP-IDF version.
For removing old versions of idf-driver, idf-python-wheels use command 'python.exe C:\MAJOR\esp\Espressif\frameworks\esp-idf-v5.2.1\tools\idf_tools.py uninstall'
For free up even more space, remove installation packages of those tools. Use option 'python.exe C:\MAJOR\esp\Espressif\frameworks\esp-idf-v5.2.1\tools\idf_tools.py uninst
all --remove-archives'.

Done! You can now compile ESP-IDF projects.
Go to the project directory and run:

idf.py build

C:\MAJOR\esp\Espressif\frameworks\esp-idf-v5.2.1>
```

2. Example program to check whether the board working or not

Hello world

Program :

```
#include <stdio.h>
#include
<inttypes.h>
#include
"sdkconfig.h"
#include
"freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_chip_info.h"
#include "esp_flash.h"
#include "esp_system.h"

void app_main(void)
{
    printf("Hello
    world!\n");

    /* Print chip information
    */esp_chip_info_t
    chip_info; uint32_t
    flash_size;
    esp_chip_info(&chip_inf
    o);
    printf("This is %s chip with %d CPU core(s),
    %s%s%s%s, ", CONFIG_IDF_TARGET,
    chip_info.cores,
    (chip_info.features & CHIP_FEATURE_WIFI_BGN) ?
    "WiFi/" : "", (chip_info.features & CHIP_FEATURE_BT) ?
    "BT" : "", (chip_info.features & CHIP_FEATURE_BLE) ?
    "BLE" : "", (chip_info.features &
    CHIP_FEATURE_IEEE802154) ? ", 802.15.4
```

```
(Zigbee/Thread)" : "");
```

```
    unsigned major_rev = chip_info.revision / 100;
    unsigned minor_rev = chip_info.revision % 100;
    printf("silicon revision v%d.%d, ", major_rev,
        minor_rev);if(esp_flash_get_size(NULL,
        &flash_size) != ESP_OK) {
        printf("Get flash size failed");
        return;
    }

    printf("%" PRIu32 "MB %s flash\n", flash_size / (uint32_t)(1024
        * 1024),(chip_info.features &
        CHIP_FEATURE_EMB_FLASH) ? "embedded" :
        "external");

    printf("Minimum free heap size: %" PRIu32 "
        bytes\n",esp_get_minimum_free_heap_size());

    for (int i = 10; i >= 0; i--) {
        printf("Restarting in %d
            seconds...\n", i);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
    printf("Restarting
        now.\n");fflush(stdout);
    esp_restart();
}
```


LED blinking

Program :

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "led_strip.h"
#include "sdkconfig.h"

static const char *TAG =

"example";#define

BLINK_GPIO 4;

static uint8_t s_led_state = 0;

#ifdef

CONFIG_BLINK_LED_STR

IP

static led_strip_handle_t led_strip;

static void blink_led(void)
{
    /* If the addressable LED is
    enabled */if (s_led_state) {
        /* Set the LED pixel using RGB from 0 (0%) to 255 (100%) for
        each color */led_strip_set_pixel(led_strip, 0, 16, 16, 16);
        /* Refresh the strip to send data
        */led_strip_refresh(led_strip);
```

```

    } else {
        /* Set all LED off to clear all
        pixels */
        led_strip_clear(led_strip);
    }
}

static void configure_led(void)
{
    ESP_LOGI(TAG, "Example configured to blink addressable LED!");
    /* LED strip initialization with the GPIO and pixels number*/
    led_strip_config_t strip_config = {
        .strip_gpio_num = BLINK_GPIO,
        .max_leds = 1, // at least one LED on board
    };
    #if CONFIG_BLINK_LED_STRIP_BACKEND_RMT
        led_strip_rmt_config_t rmt_config = {
            .resolution_hz = 10 * 1000 * 1000, // 10MHz
            .flags.with_dma = false,
        };
        ESP_ERROR_CHECK(led_strip_new_rmt_device(&strip_config,
            &rmt_config, &led_strip));
    #elif CONFIG_BLINK_LED_STRIP_BACKEND_SPI
        led_strip_spi_config_t spi_config = {
            .spi_bus = SPI2_HOST,
            .flags.with_dma = true,
        };
        ESP_ERROR_CHECK(led_strip_new_spi_device(&strip_config,
            &spi_config, &led_strip));
    #else
        #error "unsupported LED strip
        backend"#endif
        /* Set all LED off to clear all
        pixels */
        led_strip_clear(led_strip);

```

```

}

#elif CONFIG_BLINK_LED_GPIO

static void blink_led(void)
{
    /* Set the GPIO level according to the state (LOW or
    HIGH)*/gpio_set_level(BLINK_GPIO, s_led_state);
}

static void configure_led(void)
{
    ESP_LOGI(TAG, "Example configured to blink
    GPIO LED!");gpio_reset_pin(BLINK_GPIO);
    /* Set the GPIO as a push/pull output */
    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
}

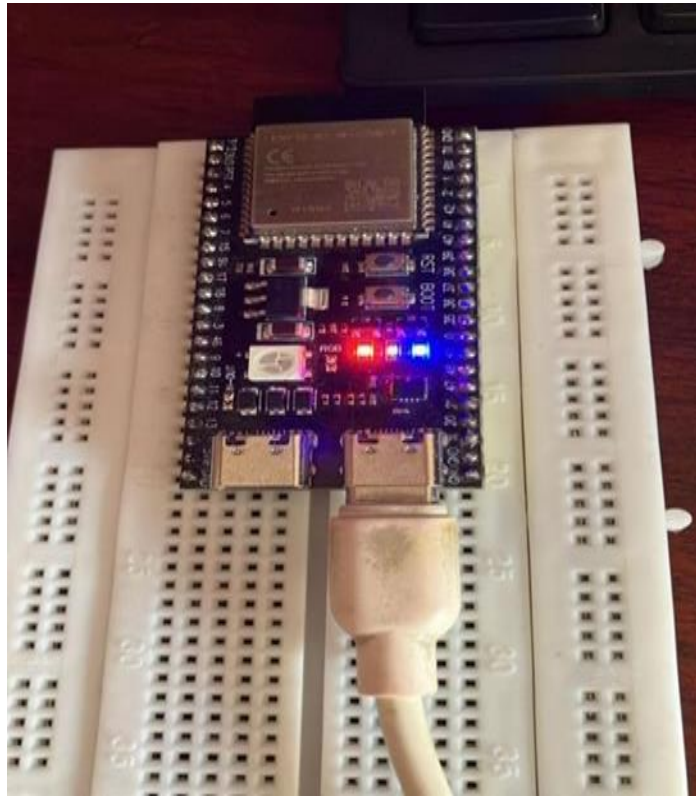
#else
#error "unsupported LED
type"#endif

void app_main(void)
{

    configure_led(

);while (1) {
    ESP_LOGI(TAG, "Turning the LED %s!", s_led_state == true ? "ON" :
    "OFF");blink_led();
    /* Toggle the LED state
    */ s_led_state =
    !s_led_state;
    vTaskDelay(5000 / portTICK_PERIOD_MS);
}
}

```



```
I (283) sleep: Configure to isolate all GPIO pins in sleep state
I (289) sleep: Enable automatic switching of GPIO sleep configuration
I (297) main_task: Started on CPU0
I (307) main_task: Calling app_main()
I (307) example: Example configured to blink addressable LED!
I (317) gpio: GPIO[48]| InputEn: 0| OutputEn: 1| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:0
I (317) example: Turning the LED OFF!
I (5327) example: Turning the LED ON!
I (10327) example: Turning the LED OFF!
I (15327) example: Turning the LED ON!
I (20327) example: Turning the LED OFF!
I (25327) example: Turning the LED ON!
I (30327) example: Turning the LED OFF!
```

Figure j: LED BLINKING

3.creation of the project

The idf.py create-project command is a powerful tool provided by the Espressif IoT Development Framework (ESP-IDF) to streamline the process of initializing new projects for ESP32-based development. When we run the idf.py create-project command, it generates a new project directory with a standard layout, including essential configuration files, example source code, and build scripts.

```
C:\MAJOR\esp>idf.py create-project hello
Executing action: create-project
The project was created in C:\MAJOR\esp\hello
```

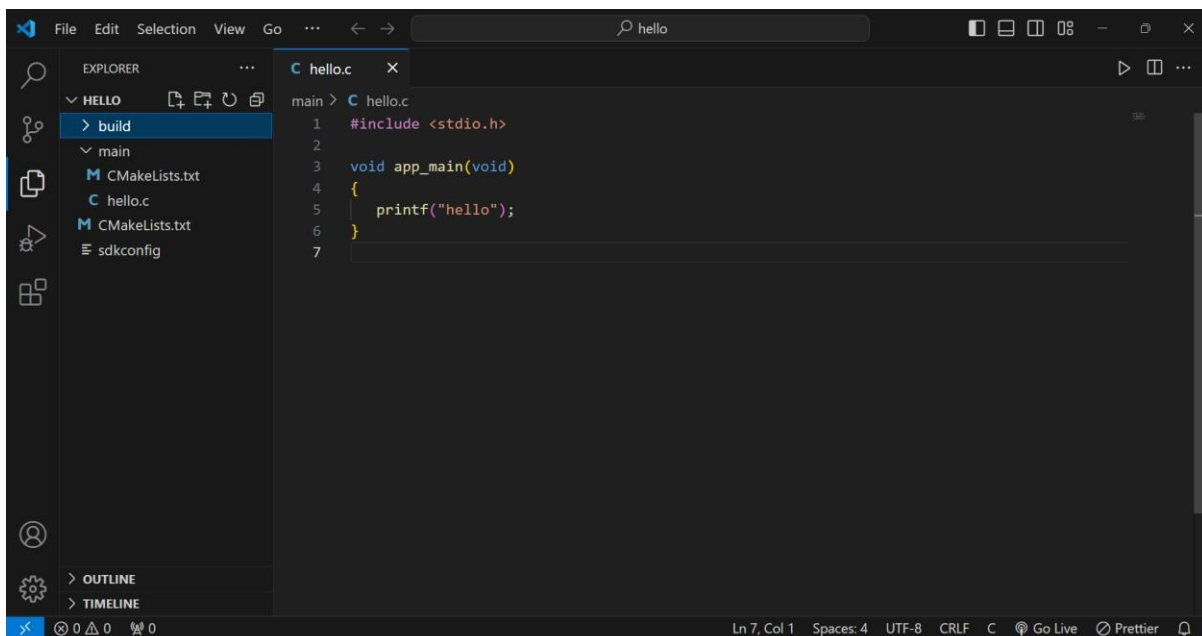


Figure k:Structure of Project

V. CODES OF SYNCHRONIZATION

```
#include <stdio.h>
#include "driver/i2c.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "sdkconfig.h"
#include "dht11.h"
#include "max30100.h"

#define I2C_SDA 8
#define I2C_SCL 9
#define I2C_FRQ 100000
#define I2C_PORT 0

max30100_config_t max30100 = {}; // Initialize the max30100 configuration

esp_err_t i2c_master_init(i2c_port_t i2c_port)
{
    i2c_config_t conf = {}; // Initialize I2C configuration
    conf.mode = I2C_MODE_MASTER; //set mode of I2C
    conf.sda_io_num = I2C_SDA; //set SDA Pin
    conf.scl_io_num = I2C_SCL; //set SCL Pin
    conf.sda_pullup_en = GPIO_PULLUP_ENABLE; //set pull resistors
    conf.scl_pullup_en = GPIO_PULLUP_ENABLE;
    conf.master.clk_speed = I2C_FRQ; //set clock frequency
    i2c_param_config(i2c_port, &conf); //configure all parameters of I2C
    return i2c_driver_install(i2c_port, I2C_MODE_MASTER, 0, 0, 0); //install I2C drivers
}

void get_bpm(void* param)
{
    printf("MAX30100 Test\n");
    max30100_data_t result = {}; //intialize max30100 data
    while(1)
```

```

{
    printf("MAX30100 task Priority : %d \n",uxTaskPriorityGet(NULL));
    // Update sensor, saving to "result"
    esp_err_t ret = max30100_update(&max30100, &result);
    if (ret != ESP_OK) //check return value is error
    {
        //esp_err_to_name(ret) is to Convert ESP-IDF error code to its string representation
        /*"ESP_OK" for no error.
        "ESP_ERR_NO_MEM" if there is an out of memory error.
        "ESP_ERR_INVALID_ARG" if an invalid argument is passed.
        "ESP_FAIL" for a generic failure.
        "ESP_ERR_INVALID_STATE" if the operation is called in an invalid state.
        "ESP_ERR_TIMEOUT" if there is a timeout error.*/
        printf("Error updating MAX30100: %s\n", esp_err_to_name(ret));//updated error
message
    }
    else
    {
        // Print a message when sensor data is
        printf("Message : %s \n",esp_err_to_name(ret));
        printf("Sensor data updated successfully\n");
        // Print the pulse detection status
        printf("Pulse Detected: %s\n", result.pulse_detected ? "Yes" : "No");
        if(result.pulse_detected)
        {
            // Print a message when a pulse is detected
            printf("BPM: %f | SpO2: %f%%\n", result.heart_bpm, result.spO2);
        }
    }
    // Update rate: 100Hz
    vTaskDelay(1000 / portTICK_PERIOD_MS);
}
}

```

```

void dht11(void *data)

```

```

{
    while(1)
    {
        printf("DHT11 task Priority : %d \n",uxTaskPriorityGet(NULL));
        printf("Temperature is %d \n", DHT11_read().temperature);
        printf("Humidity is %d\n", DHT11_read().humidity);
        printf("Status code is %d\n", DHT11_read().status);
        vTaskDelay((5000/portTICK_PERIOD_MS));
    }
}

void app_main()
{
    DHT11_init(GPIO_NUM_4);
    //Init I2C_NUM_0
    ESP_ERROR_CHECK(i2c_master_init(I2C_PORT));
    //Init sensor at I2C_NUM_0
    ESP_ERROR_CHECK(max30100_init( &max30100, I2C_PORT,
        MAX30100_DEFAULT_OPERATING_MODE,
        MAX30100_DEFAULT_SAMPLING_RATE,
        MAX30100_DEFAULT_LED_PULSE_WIDTH,
        MAX30100_DEFAULT_IR_LED_CURRENT,
        MAX30100_DEFAULT_START_RED_LED_CURRENT,
        MAX30100_DEFAULT_MEAN_FILTER_SIZE,
        MAX30100_DEFAULT_PULSE_BPM_SAMPLE_SIZE,
        true, false ));

    //Start test task
    xTaskCreate(get_bpm, "Get BPM", 2048, NULL, 3, NULL);
    xTaskCreate(dht11, "dht11", 2048, NULL,2,NULL);
    while(1)
    {
        printf("main task priority : %d \n",uxTaskPriorityGet(NULL));
        vTaskDelay(1000/portTICK_PERIOD_MS);
    }
}

```